Практическое занятие 8

Воспроизводимый конвейер обработки данных

Цель — перенести ручные преобразования работы 7 в единый Pipeline с ColumnTransformer. Конвейер формирует 17 числовых признаков, обучает статистики только на train и сохраняет схему на validation, test и новых заказах. Базовые 16 признаков совпали с работой 7; добавлен календарный индикатор выходного дня.

Данные и допустимые входы

В orders_dirty.csv 600 строк. После удаления одного полного дубликата и одной конфликтной исторической метки осталось 598 заказов. Метка late проверена по условию actual_minutes − promised_minutes > 15. Исходный CSV не изменён; его SHA-256 проверяется после выполнения. Повторяющихся ID и пересечений между выборками нет.

Входные X: distance_km, queue_size, promised_minutes, zone, weather, created_at. courier_load отсутствует в предоставленном CSV, поэтому исключён из шаблона без выдумывания значений. vehicle_type не включён в заданный набор X. late хранится отдельно как y; order_id используется для аудита; actual_minutes применяется только для контроля исторической метки и запрещён в X.

Выборка	N	Класс 0	Класс 1	Доля класса 1
train	358	190	168	46.93%
val	120	64	56	46.67%
test	120	64	56	46.67%

Использован split_ids_06.csv из работы 7 с тем же исходным CSV: стратифицированное разбиение 60/20/20, random_state=42. Проверены уникальность ID, полнота объединения и отсутствие пересечений. Порядок строк для сравнения с ручными матрицами выровнен по order_id.

Фиксированные производные признаки

derive приводит числовые строки к числам; отрицательные значения и нулевая обещанная длительность становятся NaN. Пробельные и словесные маркеры отсутствия категорий тоже заменяются NaN. Функция не вычисляет обучаемые статистики.

Время преобразуется в sin(2πh/24) и cos(2πh/24), сохраняющие соседство 23:00 и 00:00. Плановая скорость равна 60 · distance_km / promised_minutes при положительном времени. Новый is_weekend равен 1 для субботы/воскресенья и 0 для будней; при невалидной дате — NaN. Он известен при заказе и может отражать режим нагрузки. Его полезность по test не исследовалась.

Числовая ветвь содержит 7 полей, включая 4 производных. После добавления 3 индикаторов пропусков она выдаёт 10 столбцов. Категориальная ветвь заменяет 2 поля семью One-Hot столбцами. Итого: 6 входов → 9 полей после derive → 17 выходов.

Структура конвейера и обученные параметры

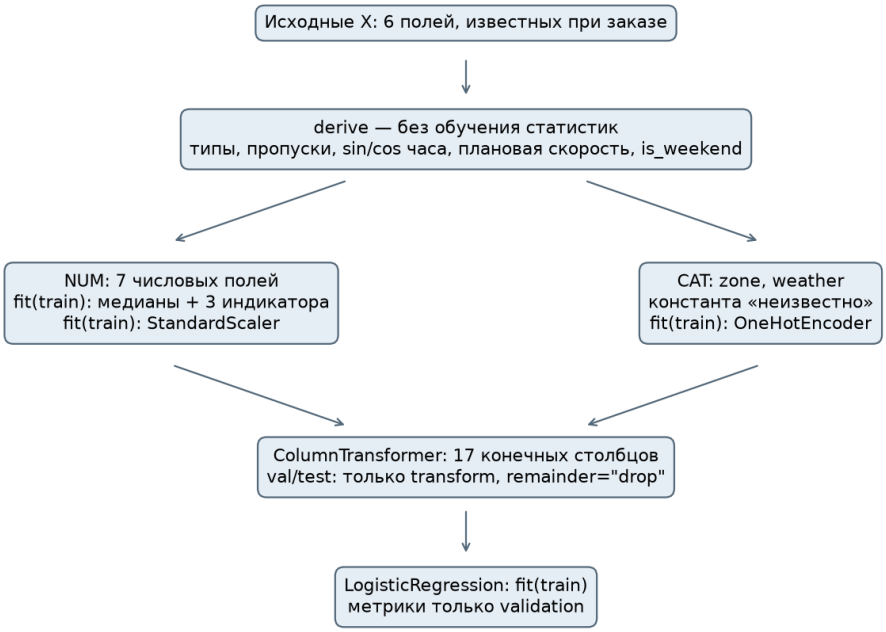


Схема 1. Последовательность шагов и потенциальные точки утечки. Обучаемые операции помечены fit(train). На validation/test применяются только сохранённые параметры. remainder="drop" исключает невыбранные колонки, но не заменяет контроль исходного X.

Признак	Медиана train	Среднее scaler	Масштаб scaler
distance_km	8.5000	8.8285	5.4792
queue_size	10.0000	10.2821	6.2397
promised_minutes	49.5000	50.0307	14.6471
hour_sin	0.0000	-0.0295	0.7093
hour_cos	-0.0000	-0.0226	0.7039
planned_speed_kmh	10.0000	9.8115	4.9803
is_weekend	0.0000	0.2793	0.4487

Медианы сверены с F_train[NUM].median(); средние и дисперсии scaler — с импутированной train-матрицей (ddof=0). Параметры общих признаков также совпали с работой 7. Три индикатора относятся к distance_km, queue_size и planned_speed_kmh; их среднее 0,002793 и масштаб 0,052778. Все параметры сохранены в 08_transform_parameters.csv.

Словари train: zone = center, east, north, south; weather = clear, rain, snow. Они сверены с уникальными значениями импутированных категорий train. Пропуски заменяются константой «неизвестно». Если она не встречалась в train, на validation она обрабатывается как новая категория. Хеш обученного конвейера не изменился после transform отложенных выборок и нового заказа.

Проверки схемы и эквивалентности

Выборка	До обработки	После обработки	База работы 7
train	358 × 6	358 × 17	358 × 16
val	120 × 6	120 × 17	120 × 16
test	120 × 6	120 × 17	120 × 16

Входные таблицы представлены в ноутбуке и сохранены как 08_input_train/val/test.csv. Готовые матрицы — 08_X_train/val/test.csv; ID и метки в том же порядке — 08_y_train/val/test.csv. Проверены сохранение числа строк, одинаковая ширина, конечность всех значений, уникальность имён и отсутствие запрещённых полей.

Для полного сравнения создан базовый Pipeline без is_weekend и обучен только на train. Эталонные ручные матрицы из reference_lb7 выровнены по ID; префиксы num__/cat__ сняты, колонки переставлены по эталону. Максимальное абсолютное различие равно 0 на train, val и test. У расширенного Pipeline все 16 общих столбцов также совпали; семнадцатый — новый календарный признак. Сравнение не использует качество модели на test для выбора настроек.

Новая __НОВАЯ_ЗОНА__ отсутствует в словаре train. Основной кодировщик с ignore выдал четыре нуля в блоке zone, сохранил остальные значения и 17 выходных полей. Отдельный кодировщик с error, обученный на тех же train-категориях, вызвал ожидаемый ValueError. Совместимость схемы не означает сохранения информации о специфике новой зоны.

Дополнительно проверены новая погода, нулевое обещанное время, невалидная дата и индикатор выходного дня на пятнице, субботе и воскресенье. Все проверки пройдены без изменения обученных параметров. Индикаторы создаются лишь для полей с пропусками на train; позднее возникшие пропуски других полей импутируются без расширения схемы.

Диагностическая модель на validation

Использована одна LogisticRegression(max_iter=1000, random_state=42) внутри полного Pipeline. Модель и её преобразования обучены только на train; порог классификации 0,5 и набор признаков заданы заранее. Validation содержит 120 заказов, из них 56 задержек (46,67%).

Balanced accuracy	F1	ROC AUC
0.689732	0.666667	0.754743

Balanced accuracy усредняет полноту двух классов; F1 учитывает precision и recall положительного класса, ROC-AUC оценивает ранжирование вероятностей. Эти результаты подтверждают работоспособность сквозного конвейера на учебных данных, но не доказывают качество на реальных заказах. Гиперпараметры не подбирались, test-score не вычислялся.

Сохранение и запуск в новом процессе

Сохранены 08_feature_pipeline.joblib и 08_demo_model.joblib. Повторная загрузка проверена в текущем ядре и отдельном Python-процессе через verify_saved.py. На 120 validation-строках преобразование совпало точно (максимальная разница 0), вероятности совпали с допуском 10^{-12}, предсказанные классы — точно. Дополнительно сверены имена столбцов и порядок ID, подготовка внутри модели и пересчитанные по сохранённым предсказаниям метрики. Результат записан в 08_reload_check.json.

Ответы на контрольные вопросы

1. fit оценивает параметры, transform применяет их без обучения, fit_transform объединяет обе операции. Медиана, среднее, масштаб и словарь оцениваются только на train; val/test проходят только transform. FunctionTransformer выполняет фиксированные вычисления, не оценивая распределение данных.

2. StandardScaler сохраняет mean_, var_, scale_ и число увиденных объектов; OneHotEncoder сохраняет categories_ и настройки обработки неизвестных значений. Поэтому повторное применение не требует вычислять статистики на новых данных.

3. FunctionTransformer располагается до ColumnTransformer, потому что сначала нужны корректные типы и вычисленные колонки, на которые ссылаются ветви. Иначе часовые координаты и плановая скорость ещё отсутствуют.

4. remainder="drop" отбрасывает неуказанные столбцы. Этого недостаточно: если actual_minutes ошибочно указан в NUM, он всё равно будет обработан. Нужна явная проверка доступных входов; ID, y и фактическая длительность не передаются в X.

5. Новое значение weather при ignore даст нули в трёх столбцах погоды; блок zone и другие признаки сохранятся. При error возникнет исключение. Чтобы выделить новой категории собственную колонку, потребуется отдельное переобучение на подходящих тренировочных данных, а не fit на текущем validation/test.

6. По test нельзя выбирать модель, регуляризацию C, порог, набор признаков, стратегию импутации или способ масштабирования. Для выбора используют validation; test сохраняют для независимой заключительной оценки.

7. Сериализованный Pipeline хранит шаги, настройки и обученные статистики, а диагностическая модель дополнительно — коэффициенты классификатора. Xt_train хранит только уже рассчитанные числа и не преобразует новые заказы. FunctionTransformer ссылается на функцию в tyumgu_feature_rules.py, поэтому этот модуль нужен при загрузке вместе с совместимыми версиями библиотек.

Воспроизводимость и вывод

Для запуска используется Python 3.12 из .venv в корне проекта; версии закреплены в requirements.txt. run_notebook.py выполняет все ячейки в новом ядре, verify_saved.py проверяет доверенные локальные файлы в отдельном процессе. Все 13 исходных ячеек, их порядок и типы сохранены. Неизвестные joblib/pkl загружать нельзя: десериализация может выполнять код.

Обработка сведена к воспроизводимому Pipeline с двумя ветвями ColumnTransformer. Статистики всех обучаемых шагов получены исключительно из train. Общие признаки совпали с ручной обработкой работы 7 после выравнивания строк и колонок. Добавленный календарный признак сохранил одинаковую размерность всех выборок. Новая категория не нарушила схему, хотя сведения о её специфике потеряны. Диагностическая модель оценена только на validation и не служит доказательством качества на реальных данных. Конвейер и модель повторно загружены в новом процессе, а исходный CSV остался неизменным.

Основные источники потенциальной утечки: fit до разделения или на val/test, включение actual_minutes/late в X, подбор параметров по test. Порядок split → fit(train) → transform(val/test), явный список X и отдельная validation-проверка устраняют эти ошибки в выполненной работе.